

UNITY PRODUCT LAB

Прогресивні завдання на прототипування ігор

6 референсних ігор | послідовні технічні кроки | UI або Random/Generation | polish

Мета роботи

Послідовно побудувати працездатний gameplay loop, а потім розширити його конкретною спеціалізованою гілкою та фінальним polish. Кожний наступний пункт виконується після перевірки попереднього.

Unity 6.x • 2D gameplay • individual progression

Як виконувати завдання

- Оберіть або отримайте від викладача одну з шести ігор-референсів.
- Виконуйте пункти 1-6 послідовно. Не переходьте далі, поки попередній пункт не працює стабільно.
- Після пункту 6 виконайте гілку А або В, призначену викладачем. Гілки мають різний фокус, але однакову роль у прогресії завдання.
- Після своєї гілки виконайте фінальні пункти 10-12.
- Не потрібно копіювати арт, баланс або точну структуру оригінальної гри. Референс визначає core loop і поведінку систем.

Правило AI: AI дозволено використовувати для однієї конкретної поточної задачі: пояснення API, debugging, приклад окремого компонента, refactoring або аналіз власного коду. Не використовуйте один prompt для генерації всієї гри або повного набору скриптів.

1. Vampire Survivors

Top-down survival: рух, автоматична атака, нескінченний тиск ворогів і нарощування сили.

Референс: [Vampire Survivors](#)

Фантазія гравця

Гравець - центр постійно зростаючого хаосу. Він не прицілюється вручну, а керує позиціонуванням, ризиком і темпом виживання.

Core loop

Move -> enemies approach -> weapon attacks automatically -> enemies die -> score/XP grows -> pressure increases.

Ключові рішення

Куди рухатися, наскільки близько підпускати ворогів, як використовувати простір арени та як розвивати параметри персонажа.

Джерела складності

Щільність ворогів, швидкість spawn, співвідношення fast/tank enemies, attack interval, damage, доступний простір для маневру.

1. Player

- Створіть Player зі SpriteRenderer, Collider2D та Rigidbody2D.
- Реалізуйте рух по X/Y через WASD або стрілки.
- Speed налаштовується через Inspector.
- Забороніть небажане фізичне обертання Player під час collisions.

Геймплейна роль: Рух є головним способом ухвалення рішень. Оскільки атака автоматична, skill expression переноситься у позиціонування: гравець створює дистанцію, шукає безпечні напрямки та уникає оточення.

2. Enemy

- Створіть Enemy prefab зі SpriteRenderer і Collider2D.
- Enemy автоматично рухається до Player.
- Movement Speed налаштовується через Inspector.

Геймплейна роль: Простий pursuit формує постійний spatial pressure. Навіть примітивний ворог працює, якщо кількість і геометрія підходу змушують гравця постійно змінювати позицію.

3. Health та Damage

- Додайте Health для Enemy та Player.
- Реалізуйте отримання damage.
- Enemy зникає при HP ≤ 0 .
- Enemy при контакті завдає Player damage.
- Player програє при HP ≤ 0 .

Геймплейна роль: Health перетворює collision на керований ризик, а не миттєву помилку. Це дозволяє створити напругу, recovery та балансувати aggressiveness ворогів через damage, а не лише через швидкість.

4. Automatic Weapon

- Створіть Projectile prefab.
- Projectile автоматично створюється через заданий interval.
- Projectile летить до найближчого Enemy.
- При попаданні завдає damage і зникає після hit або lifetime.
- Damage, Projectile Speed та Attack Interval налаштовуються через Inspector.

Геймплейна роль: Автоматична зброя прибирає необхідність постійно прицілюватися і звільняє увагу для movement. Attack interval і damage створюють зрозумілу power curve: гравець відчуває посилення через частоту та результативність атак.

5. Enemy Spawner

- Створіть автоматичний Enemy Spawner.
- Enemies з'являються навколо Player, але не безпосередньо на ньому.
- Додайте Spawn Interval.
- Додайте мінімальну/максимальну дистанцію або spawn area.

Геймплейна роль: Spawner перетворює набір окремих ворогів на survival-систему. Найважливіше - контролювати темп тиску: занадто рідкісний spawn не створює напруги, занадто близький або частий робить damage випадковим і несправедливим.

6. Gameplay Loop

- Додайте Score або Kill Counter.
- За смерть Enemy нараховуються points.
- Додайте Game Over після смерті Player.
- Додайте Restart.
- При запуску гри enemies створюються автоматично.

Геймплейна роль: Score дає короткострокову мету, а Game Over закриває цикл і робить кожну спробу вимірною. Навіть без progression гра вже має rhythm: виживання -> ризик -> результат -> нова спроба.

ГІЛКА А - UI TA PROGRESSION

7A. HUD

- Додайте HP Bar.
- Додайте Score.
- Додайте Timer тривалості гри.

Геймплейна роль: HUD повинен відповідати на три питання без відкривання меню: скільки ресурсу життя залишилося, наскільки успішна спроба і як довго гравець уже виживає. Це підтримує рішення у реальному часі.

8A. XP

- Після смерті Enemy з'являється XP pickup.
- Player збирає XP.
- Додайте XP Bar.
- Після заповнення XP Bar збільшуйте Level.

Геймплейна роль: XP переводить kills у довгостроковий reward і створює ризик: щоб отримати нагороду, гравець має фізично підійти до pickup. Це змушує балансувати безпеку та жадібність.

9A. Level Up

- Після нового Level поставте gameplay на pause.
- Покажіть три upgrade-кнопки: +Damage, +Attack Speed, +Movement Speed.
- Після вибору upgrade продовжте gameplay.

Геймплейна роль: Level-up pause створює короткий tactical break і дає meaningful choice. Три апгрейди змінюють різні сторони loop: damage - ефективність, attack speed - темп, movement - контроль простору.

ГІЛКА В - RANDOM TA DIFFICULTY

7B. Spawn Distribution

- Spawn відбувається у випадкових позиціях навколо Player.
- Задайте мінімальну та максимальну дистанцію spawn.

Геймплейна роль: Рандомізація позицій робить кожну хвилину непередбачуваною, але min/max distance зберігає fairness: ворог не повинен з'являтися без можливості реакції.

8B. Enemy Types

- Створіть Normal, Fast і Tank Enemy prefabs.
- Spawner випадково вибирає тип Enemy.

Геймплейна роль: Три архетипи створюють різні просторові задачі: Normal задає базовий ритм, Fast карає пасивність, Tank довше блокує напрямом і накопичує crowd pressure.

9B. Difficulty Progression

- З часом зменшуйте Spawn Interval.
- Збільшуйте шанс Fast і Tank enemies.
- Збільшуйте кількість активних enemies.

Геймплейна роль: Складність повинна рости не одним параметром, а комбінацією частоти, складу та щільності. Це створює відчуття ескалації без різкого стрибка.

10. Hit Feedback

- Додайте щонайменше два ефекти: flash/color change, knockback, scale punch або коротку hit animation.

Геймплейна роль: Hit feedback підтверджує причинно-наслідковий зв'язок між атакою та результатом. Гравець має відчуття попадання до того, як прочитає зміну HP.

11. Death Polish

- Додайте Particle System.
- Додайте animation або DOTween effect.
- Додайте sound effect.

Геймплейна роль: Смерть ворога - найчастіша reward-подія. Її візуальна та звукова виразність визначає, наскільки приємно повторювати основну дію гри сотні разів.

12. Advanced System

- Реалізуйте один варіант: Object Pool для projectiles; ScriptableObject для Weapon parameters; ScriptableObject для Enemy parameters; event-based Score; event-based Enemy Death.

Геймплейна роль: Зробіть систему масштабованою: новий weapon, enemy або reaction на death мають додаватися без переписування існуючого gameplay.

2. Galaga

Fixed shooter: горизонтальний рух, точна стрільба, formation enemies та хвилі.

Референс: [Gameplay reference](#)

Фантазія гравця

Core loop

Ключові рішення

Джерела складності

Гравець керує невеликим кораблем під тиском організованої ворожої формації та намагається знищити її, не втративши контроль над вузькою безпечною зоною.

Move horizontally -> shoot -> destroy enemies -> dodge fire -> clear formation -> next wave.

Коли стріляти, де стояти відносно formation, коли ризикувати заради kill та як ухилятися від ворожих projectile patterns.

Щільність формації, fire rate ворогів, projectile speed, рух formation, кількість HP/lives та composition наступних waves.

1. Player Ship

- Створіть Player Ship зі SpriteRenderer і Collider2D.
- Реалізуйте horizontal movement.
- Player не виходить за межі gameplay area.
- Movement Speed налаштовується через Inspector.

Геймплейна роль: Обмеження руху однією віссю робить positioning дуже читабельним: кожний крок ліворуч/праворуч одночасно є і aim correction, і dodge. Це дозволяє складності виникати з pattern-ів, а не зі складного керування.

2. Shooting

- Створіть Projectile prefab і Spawn Point.
- Реалізуйте Fire Input.
- Додайте cooldown.
- Projectile рухається вгору.
- Projectile видаляється за межами gameplay area або після lifetime.

Геймплейна роль: Cooldown задає rhythm натискань і обмежує DPS. Якщо постріл можна spam-ити без обмежень, positioning і timing втрачають значення.

3. Enemy

- Створіть Enemy prefab з Collider2D і Health.
- Projectile завдає damage.
- Enemy знищується при HP ≤ 0 .
- За kill нараховуються points.

Геймплейна роль: Окремий enemy - мінімальна ціль із чітким feedback. HP дозволяє змішувати одно- та багатохітові targets, а score підсилює бажання чистити formation максимально ефективно.

4. Enemy Formation

- Створіть formation мінімум 3 x 4 enemies.
- Enemies є частиною спільної formation.

- Formation рухається горизонтально.
- Біля меж змінює напрямок.

Геймплейна роль: Formation перетворює ворогів на один великий рухомий патерн. Вона змушує гравця читати простір наперед і створює синхронний rhythm, якого немає у випадковому русі окремих enemies.

5. Enemy Attack

- Enemies періодично стріляють вниз.
- Створіть Enemy Projectile prefab.
- Player отримує damage.
- Player має мінімум 3 HP або 3 lives.

Геймплейна роль: Ворожі постріли змушують чергувати attack і dodge. Lives роблять помилку відчутною, але не завершують run миттєво, тому гравець може адаптуватися до pattern.

6. Gameplay Loop

- Додайте Score.
- Додайте Win після знищення всіх enemies.
- Додайте Game Over після смерті Player.
- Додайте Restart.

Геймплейна роль: Clear condition створює конкретну коротку мету: поле має бути очищене. Це відрізняє Galaga від endless survival і робить progress візуально очевидним - formation фізично рідшає.

ГІЛКА А - UI

7A. HUD

- Показуйте Score.
- Показуйте HP/Lives.
- Показуйте Remaining Enemies.

Геймплейна роль: Remaining Enemies робить прогрес до перемоги явним, а lives - ризик помилки. HUD повинен бути компактним і не конкурувати з projectile-heavy gameplay.

8A. Start Screen

- Додайте назву гри.
- Додайте Play button.
- Додайте коротку інструкцію.
- Gameplay не починається до Play.

Геймплейна роль: Start screen створює зрозумілий entry state і дає гравцеві секунду сформувати mental model до початку pressure.

9A. Result Screen

- Покажіть Win/Game Over.
- Покажіть Final Score.
- Додайте Restart.
- Анімуйте появу panel через Animator або DOTween.

Геймплейна роль: Result screen закриває run і переводить outcome у reward/feedback. Анімація робить зміну стану від gameplay до result читаємою і навмисною.

ГІЛКА В - RANDOM TA GENERATION

7B. Enemy Types

- Створіть Basic Enemy, Fast Enemy і Tough Enemy.

Геймплейна роль: Архетипи змінюють пріоритет цілей: Fast швидше змінює простір, Tough довше займає слот formation, Basic підтримує ритм.

8B. Random Formation

- При створенні formation вибирайте тип Enemy випадково.
- Задайте різні probabilities для трьох типів.

Геймплейна роль: Weighted random дозволяє варіювати склад formation без втрати контролю. Дизайнер визначає не конкретний результат, а розподіл можливих encounters.

9B. Generated Waves

- Після знищення formation створюйте наступну.
- Випадково визначайте rows, columns та склад enemies.
- З кожною wave збільшуйте movement speed.

Геймплейна роль: Генерація хвиль додає replayability, а гарантоване зростання швидкості створює macro-progression: навіть вдала випадкова wave не повинна зупинити ескалацію.

10. Explosion

- При смерті Enemy додайте Particle System.
- Додайте animation або DOTween effect.
- Додайте sound.

Геймплейна роль: Explosion є винагородою за точне попадання і очищення простору. У shooter вона також допомагає миттєво відрізнити successful hit від miss.

11. Shooting Polish

- Додайте muzzle flash.
- Додайте projectile trail.
- Додайте shooting sound.
- Додайте hit feedback.

Геймплейна роль: Стрільба - найчастіша дія гравця. Якщо вона має слабкий feedback, гра відчувається порожньою навіть при правильній механіці.

12. Object Pool

- Замініть Instantiate/Destroy для Player projectiles на Object Pool.

Геймплейна роль: Pooling тут виправданий самим gameplay: projectiles створюються постійно. Це хороший приклад оптимізації, яка впливає з частоти конкретної дії, а не додається абстрактно.

3. Plants vs. Zombies

Lane defense: grid placement, економіка Sun, автоматичні атаки та хвилі ворогів.

Референс: [Plants vs. Zombies - EA](#)

Фантазія гравця

Core loop

Ключові рішення

Джерела складності

Гравець не керує одним героєм. Він будує систему захисту, інвестує обмежений ресурс і спостерігає, як його рішення витримують тиск хвилі.

Gain Sun -> choose plant -> place on grid -> plants act automatically -> zombies pressure lanes -> survive wave.

Куди інвестувати Sun, який lane потребує відповіді, коли економити ресурс, коли ставити damage, defense або economy unit.

Темп Sun generation, plant costs, zombie HP/speed, lane distribution, wave composition, cooldown між placements.

1. Grid

- Створіть gameplay grid мінімум 5 rows x 7 columns.
- Кожна cell має визначену позицію і знає, чи вона вільна.
- Візуально покажіть структуру grid.

Геймплейна роль: Grid перетворює простір на дискретні слоти й робить кожне placement рішенням із opportunity cost: одна cell не може одночасно виконувати дві ролі.

2. Plant Placement

- Створіть перший Plant prefab.
- При натисканні на вільну cell Plant з'являється у її центрі.
- Не дозволяйте ставити дві plants в одну cell.

Геймплейна роль: Placement є основною player action. Важливо, щоб вона була точною, передбачуваною та читабельною - помилка input не повинна виглядати як стратегічна помилка.

3. Zombie

- Створіть Zombie prefab.
- Zombie з'являється справа.
- Zombie належить одному lane.
- Рухається справа наліво.
- Має Health.

Геймплейна роль: Lane прив'язка спрощує threat evaluation: гравець одразу бачить, який ряд потребує відповіді. Це дозволяє побудувати стратегічну складність без складного pathfinding.

4. Shooter Plant

- Shooter Plant автоматично створює projectile.
- Projectile рухається вправо.
- При попаданні завдає damage Zombie.
- Projectile зникає після hit.

- Zombie зникає при HP ≤ 0 .

Геймплейна роль: Автоматична атака робить placement головним рішенням. Після того як plant поставлена, гравець оцінює результат свого попереднього рішення, а не micromanage кожен постріл.

5. Zombie Attack

- Коли Zombie доходить до Plant, він зупиняється.
- Zombie атакує Plant через interval.
- Plant має Health.
- Після смерті Plant Zombie продовжує рух.

Геймплейна роль: Це створює frontline: plants не лише завдають damage, а й купують час. Health перетворює відстань і витривалість на стратегічний ресурс.

6. Resources та Gameplay Loop

- Додайте ресурс Sun.
- Кожен Plant має Cost.
- Placement дозволений лише при достатньому Sun.
- Після placement віднімайте Cost.
- Sun автоматично генерується.
- Автоматично spawn zombies.
- Zombie на лівому краю -> Game Over.
- Після завершення всіх enemies -> Win.
- Додайте Restart.

Геймплейна роль: Економіка Sun створює tension між негайним захистом і майбутньою ефективністю. Кожне placement стає не просто дією, а витратою обмеженого ресурсу з наслідками для наступних секунд.

ГІЛКА А - UI

7A. Plant Selection

- Створіть UI для Shooter Plant, Wall Plant та Sun Plant.

Геймплейна роль: Три ролі утворюють мінімальну strategy triangle: damage, defense, economy. UI має зробити ці ролі зрозумілими ще до placement.

8A. Plant Buttons

- Кожна кнопка показує icon і Cost.
- Якщо Sun недостатньо, кнопку не можна використати.

Геймплейна роль: Disabled state є важливим affordance: гравець повинен бачити не лише що доступно, а й чому конкретна дія зараз неможлива.

9A. Gameplay UI

- Додайте Sun Counter.
- Додайте Wave Progress.
- Додайте Win Panel, Game Over Panel і Restart.

Геймплейна роль: Wave Progress дає часовий контекст: одна й та сама кількість Sun має різну цінність на початку і наприкінці хвилі. UI допомагає гравцеві оцінювати ризик інвестиції.

ГІЛКА В - RANDOM TA WAVES

7B. Random Lane

- Zombie spawn відбувається у випадково вибраному lane.

Геймплейна роль: Random lane selection змушує будувати гнучкий defense замість memorized solution. Водночас кількість lanes обмежує entropy і залишає ситуацію читабельною.

8B. Zombie Types

- Створіть Normal Zombie, Fast Zombie та Tank Zombie.
- Spawner вибирає тип за probabilities.

Геймплейна роль: Архетипи змінюють потрібну відповідь: Fast створює терміновість, Tank перевіряє sustained damage, Normal заповнює базовий ритм.

9B. Difficulty Progression

- Кожна wave містить більше zombies.
- Spawn Interval зменшується.
- Шанс Fast і Tank Zombie збільшується.

Геймплейна роль: Комбіноване масштабування створює ескалацію і перевіряє, чи побудована гравцем економіка та оборона справді масштабуються разом із загрозою.

10. Animation

- Додайте animation або DOTween для Plant attack, Zombie hit та Plant placement.

Геймплейна роль: Placement, attack і hit - три ключові події loop. Якщо кожна має окремий motion language, стан системи читається навіть без тексту.

11. Death/Hit Effects

- Додайте Zombie hit feedback.
- Додайте death animation/effect.
- Додайте sound.

Геймплейна роль: У defense game гравець часто спостерігає за автоматичними діями. Feedback компенсує відсутність прямого control і робить результат build-рішень відчутним.

12. ScriptableObject

- Винесіть параметри Plants або Zombies у ScriptableObject.

Геймплейна роль: Data-driven unit definitions дозволяють балансувати економию і combat без зміни коду та перетворюють додавання нового юніта на контентну, а не архітектурну задачу.

4. Pac-Man

Maze chase: grid movement, collectibles, lives та переслідування ghosts.

Референс: [Gameplay reference](#)

Фантазія гравця

Core loop

Ключові рішення

Джерела складності

Гравець постійно балансує між жадібністю - зібрати ще кілька pellets - і безпекою - залишити собі шлях відступу від ghosts.

Navigate maze -> collect pellets -> read enemy routes -> avoid contact -> clear maze.

Який маршрут обрати, чи йти у dead end за reward, коли ризикувати заради special pellet і як використовувати intersections для втечі.

Швидкість ghosts, topology maze, кількість safe intersections, розташування pellets, lives, тривалість vulnerable state.

1. Maze

- Створіть corridors, walls, intersections і dead ends.
- Зробіть щонайменше два альтернативні маршрути між частинами maze.

Геймплейна роль: Maze є головною системою level design. Intersections створюють рішення, dead ends - ризик, альтернативні маршрути - можливість планувати втечу.

2. Player Movement

- Реалізуйте рух у чотирьох напрямках.
- Player не проходить крізь walls.
- Movement відповідає структурі maze/grid.

Геймплейна роль: Дискретні напрямки роблять input простим, але рішення частими. Читабельність поворотів важливіша за фізичну реалістичність.

3. Pellets

- Розмістіть мінімум 30 pellets.
- Pellet зникає після збору.
- За кожен Pellet додавайте Score.
- Зберігайте Remaining Pellets.

Геймплейна роль: Pellets одночасно є score, navigation breadcrumbs і progress indicator. Вони змушують гравця відвідувати небезпечні частини maze, а не просто ховатися.

4. Ghost

- Створіть Ghost prefab.
- Ghost рухається між waypoints.
- Не проходить крізь walls.
- Створіть маршрут із декількох waypoint points.

Геймплейна роль: Навіть deterministic patrol вже створює stealth-like timing puzzle: гравець може спостерігати, прогнозувати і вибрати момент проходку.

5. Lives

- При collision з Ghost зменшуйте lives.
- Поверніть Player у стартову позицію.
- Після втрати всіх lives покажіть Game Over.

Геймплейна роль: Lives дозволяють помилятися, але зберігають ціну ризику. Reset position створює rhythm між напруженим chase і коротким відновленням.

6. Win

- Після збору останнього Pellet покажіть Win.
- Зупиніть gameplay.
- Додайте Restart.

Геймплейна роль: Умова перемоги прив'язана до повного освоєння простору: гравець не може уникати небезпечних зон назавжди, бо кожна частина maze містить progress.

ГІЛКА А - UI

7A. HUD

- Показуйте Score.
- Показуйте Lives.
- Показуйте Remaining Pellets.

Геймплейна роль: HUD робить cost помилки і progress до clear прозорими. Remaining Pellets особливо важливий наприкінці рівня, коли в maze залишаються одиничні цілі.

8A. Start Countdown

- Покажіть 3 -> 2 -> 1 -> GO.
- До завершення countdown Player і Ghosts не рухаються.

Геймплейна роль: Countdown синхронізує attention гравця з початком chase і прибирає unfair старт, коли ворог уже рухається до того, як гравець готовий.

9A. Result Screen

- Покажіть Win/Game Over.
- Покажіть Final Score.
- Додайте Restart.
- Анімуйте появу panel.

Геймплейна роль: Result screen фіксує outcome і підсилює завершеність run. Це особливо важливо у коротких arcade loops.

ГІЛКА В - RANDOM

7B. Ghost Movement

- На intersection Ghost випадково обирає один із допустимих напрямків.

Геймплейна роль: Рандомний вибір руйнує повністю memorized route, але обмеження лише допустимими напрямками зберігає поведінку правдоподібною і керованою.

8B. Multiple Ghosts

- Створіть мінімум 3 Ghosts.
- Задайте пізні Movement Speed та Starting Positions.

Геймплейна роль: Кілька переслідувачів створюють overlapping threats. Навіть без складного AI різні стартові позиції й швидкості формують непередбачувані перетини маршрутів.

9B. Automatic Collectible Placement

- Створюйте pellets автоматично у доступних grid cells.
- Не створюйте pellets всередині walls.
- Випадково замініть декілька pellets на special pellets.

Геймплейна роль: Автоматичне placement відділяє gameplay rules від конкретного level layout. Special pellets створюють hotspots - точки, заради яких гравець свідомо приймає підвищений ризик.

10. Power Pellet

- Після Special Pellet Ghosts тимчасово переходять у Vulnerable state.
- Player може знищити Ghost при контакті.
- Vulnerable state завершується через заданий час.

Геймплейна роль: Power Pellet на короткий час інвертує ролі predator/prey. Така зміна правил різко підвищує динаміку: гравець переходить від уникнення до агресії, але мусить контролювати timer.

11. Visual States

- Ghost візуально відрізняється у Normal і Vulnerable states.
- Додайте animation або зміну sprite/color.

Геймплейна роль: State change має бути зрозумілий без читання UI. Візуальний стан є gameplay information: він прямо визначає, чи collision небезпечний чи вигідний.

12. Polish

- Додайте звуки збору.
- Додайте Power Pellet effect.
- Додайте Ghost death effect.
- Додайте animation або DOTween для UI/result.

Геймплейна роль: Рас-Ман побудований на частих дрібних reward events. Звук і короткі ефекти перетворюють монотонний збір у ритмічну послідовність підтверджень.

5. Hungry Shark

2D survival/exploration: вільний рух, hunger pressure, їжа, хижаки та growth.

Референс: [Hungry Shark Evolution - Ubisoft](#)

Фантазія гравця

Core loop

Ключові рішення

Джерела складності

Гравець - хижак, який мусить постійно рухатися й харчуватися. Світ одночасно є джерелом reward і небезпеки.

Explore -> find food -> eat -> restore hunger -> avoid predators/hazards -> grow -> survive longer.

Куди плисти, чи безпечна конкретна ціль, чи варто заходити у небезпечну область за цінною їжею, коли відступити.

Швидкість hunger decay, density food, співвідношення food/predators, movement speed, damage, size thresholds, доступність safe zones.

1. Shark

- Створіть Shark Player зі SpriteRenderer, Rigidbody2D і Collider2D.
- Реалізуйте movement по X/Y.
- Shark повертається або flip-иться за напрямком руху.

Геймплейна роль: Вільний 2D movement створює fantasy плавання і дає змогу будувати просторові ризики у будь-якому напрямку, а не лише на одній осі.

2. Camera

- Реалізуйте camera follow.
- Gameplay area більша за один екран.
- Camera не повинна миттєво стрибати при русі Player.

Геймплейна роль: Камера визначає, скільки інформації про небезпеку бачить гравець. Smooth follow підтримує відчуття інерції й не перетворює різкий input на візуальний дискомфорт.

3. Fish

- Створіть Fish prefab з Collider2D.
- Shark може її з'їсти.
- Після eating Fish зникає і Score збільшується.

Геймплейна роль: Їжа - найчастіша reward-подія. Вона повинна бути легко розпізнаваною і достатньо частою, щоб гравець постійно планував наступну коротку ціль.

4. Hunger

- Додайте Hunger.
- Hunger постійно зменшується.
- Eating Fish відновлює Hunger.
- При Hunger = 0 Shark починає втрачати HP.

Геймплейна роль: Hunger не дозволяє пасивно стояти у безпечному місці. Він перетворює exploration з опції на необхідність і створює постійний soft timer.

5. Danger

- Створіть Hazard або Predator.
- При контакті Shark отримує damage.
- Dangerous object не працює як звичайна edible Fish.

Геймплейна роль: Контраст між edible і dangerous objects створює основну систему розпізнавання ризику. Гравець повинен оцінювати не лише відстань до цілі, а й її категорію.

6. Gameplay Loop

- Автоматично spawn Fish.
- Автоматично spawn hazards/predators.
- Додайте Health, Hunger і Score.
- Додайте Game Over та Restart.

Геймплейна роль: Spawner робить ecosystem постійно оновлюваною. Поєднання hunger і damage створює два типи pressure: повільний системний та миттєвий контактний.

ГІЛКА А - UI TA GROWTH

7A. HUD

- Додайте HP Bar.
- Додайте Hunger Bar.
- Додайте Score.

Геймплейна роль: HP і Hunger мають різний сенс: HP показує наслідки помилок, Hunger - майбутню загрозу. Їх не варто візуально плутати.

8A. Growth

- Eating дає Growth XP.
- Додайте Growth Bar.
- Після заповнення збільшуйте Shark Level.

Геймплейна роль: Growth перетворює повторюваний eating loop у довгостроковий progression. Навіть дрібна Fish отримує додаткову цінність як частина шляху до нового рівня.

9A. Level Up

- Після Level Up збільшіть розмір Shark.
- Збільшіть Max HP.
- Покажіть Level Up feedback.
- Анімуйте UI або Shark через DOTween/Animator.

Геймплейна роль: Level Up має змінювати не лише число, а й fantasy: більша акула візуально демонструє power growth і готує до сильніших цілей.

ГІЛКА В - RANDOM ECOSYSTEM

7B. Creature Types

- Створіть Small Fish.
- Створіть Valuable Fish.
- Створіть Predator.

Геймплейна роль: Три типи формують базову risk/reward шкалу: safe frequent reward, rare valuable target і загроза, якої треба уникати.

8B. Random Spawn

- Для нового creature випадково визначайте type, spawn position, movement direction і movement speed у заданому діапазоні.

Геймплейна роль: Варіативність руху й позиції робить світ живим. Діапазони важливіші за повністю хаотичні значення: randomness має створювати різні ситуації, але не ламати читабельність.

9B. Difficulty

- З часом зменшуйте кількість basic food.
- Збільшуйте шанс predators.
- Збільшуйте speed creatures.

Геймплейна роль: Складність змінює сам ecosystem: безпечні ресурси стають рідшими, а небезпека - частішою. Це відчувається природніше, ніж просте збільшення HP ворогів.

10. Shark Animation

- Додайте Swimming Animation.
- Додайте Bite Animation.
- Можна використовувати Animator або Spine.

Геймплейна роль: Swimming підтримує постійний motion fantasy, а Bite має синхронізуватися з моментом reward. Хороша анімація тут безпосередньо підсилює відчуття контролю та сили.

11. Environment Effects

- Додайте щонайменше два ефекти: bubbles, eating particles, hit particles, trail, camera shake, underwater ambience.

Геймплейна роль: Ambient effects продають середовище, а interaction effects - наслідки дій. Разом вони відділяють просто рух sprite по синьому фону від відчуття підводного простору.

12. Reusable Creature System

- Реалізуйте один варіант: IEatable; IDamageable; ScriptableObject для Creature parameters.

Геймплейна роль: Reusable creature rules дозволяють швидко будувати food chain. Критерій: новий вид має додаватися через data/prefab і мінімум нового gameplay code.

6. Swamp Attack

Defense shooter: aim, weapons, enemies, база під тиском і хвильова ескалація.

Референс: [Swamp Attack - Google Play](#)

Фантазія гравця

Core loop

Ключові рішення

Джерела складності

Гравець тримає позицію і захищає її вогневою силою. Простір простіший, ніж у top-down action, тому depth виникає з target priority, fire rate, ammo та waves.

Enemies spawn -> approach base -> player aims/shoots -> enemies die or damage base -> next pressure cycle.

Кого стріляти першим, коли витратити ammo, як керувати reload, коли перемикає weapon і які targets становлять найбільшу загрозу.

Enemy speed/HP, spawn density, ammo economy, reload duration, weapon spread, wave composition, base HP.

1. Base

- Створіть Base або Player Position.
- Додайте Health.
- Додайте Collider.
- Base залишається у фіксованій частині сцени.

Геймплейна роль: Фіксована база робить loss condition просторово очевидним: кожний enemy фізично наближає поразку. Це дозволяє оцінювати urgency без складного UI.

2. Enemy

- Створіть Enemy prefab.
- Додайте Collider2D та Health.
- Enemy автоматично рухається до Base.

Геймплейна роль: Рух до однієї цілі створює чисту threat line. Геймдизайнер може балансувати небезпеку через speed, HP та spawn distance, не змінюючи базову поведінку.

3. Aim

- Створіть Weapon.
- Weapon повертається у напрямку mouse cursor.
- Напрямок Projectile відповідає напрямку Weapon.

Геймплейна роль: Aim дає прямий skill component. На відміну від автоматичної атаки Vampire Survivors, тут успішність залежить від точності та target prioritization.

4. Shooting

- Створіть Projectile prefab.
- Додайте Spawn Point.
- Реалізуйте Fire Input і Cooldown.
- Projectile завдає Enemy damage.
- Enemy помирає при HP ≤ 0 .

Геймплейна роль: Cooldown створює cadence і не дозволяє weapon бути нескінченним потоком damage. Він також є головним параметром відмінності між майбутніми weapon archetypes.

5. Enemy Attack

- Коли Enemy доходить до Base, зупинить movement.
- Enemy атакує через заданий interval.
- Base отримує damage.
- Enemy не завдає damage щократно.

Геймплейна роль: Attack interval дає гравцеві коротке recovery window: enemy, який уже дістався бази, ще можна встигнути знищити. Це створює clutch-моменти замість миттєвої втрати HP.

6. Gameplay Loop

- Автоматично spawn enemies.
- За смерть Enemy нараховуйте Score.
- При Base HP ≤ 0 покажіть Game Over.
- Додайте Restart.

Геймплейна роль: Loop будується навколо захисту однієї цінності - Base HP. Score винагороджує активну оборону, а restart робить короткі спроби порівнюваними.

ГІЛКА А - UI TA WEAPONS

7A. HUD

- Показуйте Base HP.
- Показуйте Score.
- Показуйте Ammo.

Геймплейна роль: Ammo - не просто інформація, а прогноз майбутньої вразливості. Гравець повинен бачити момент, коли скоро доведеться перезаряджатися.

8A. Ammo та Reload

- Реалізуйте Magazine Size, Current Ammo, Reload, Reload Duration і візуальний Reload Indicator.
- При Ammo = 0 стріляти не можна до завершення Reload.

Геймплейна роль: Reload створює forced downtime і робить timing пострілів важливим. Гравець має уникати ситуації, коли magazine закінчується саме під час піку хвилі.

9A. Weapon Selection

- Створіть Basic Gun і Shotgun.
- Додайте UI для переключення.
- Shotgun створює декілька projectiles зі spread.

Геймплейна роль: Weapon choice має бути ситуативним: basic gun ефективний проти окремої цілі, shotgun - проти щільної групи. Якщо одна зброя завжди краща, вибір стає декоративним.

ГІЛКА В - RANDOM TA WAVES

7B. Enemy Types

- Створіть Basic Enemy.
- Створіть Fast Enemy.
- Створіть Tank Enemy.

Геймплейна роль: Ці три архетипи створюють target priority: Fast треба зупинити швидко, Tank потребує більше часу, Basic заповнює pressure між ними.

8B. Random Spawn

- Spawner випадково вибирає Enemy за probability.

Геймплейна роль: Weighted random створює варіативність без повної непередбачуваності. Це дозволяє контролювати середню складність і при цьому уникати однакових хвиль.

9B. Wave Generator

- Кожна wave має випадкову кількість enemies у заданому діапазоні.
- Spawn Interval випадковий у заданому діапазоні.
- Склад enemies відрізняється.
- На наступних waves підвищуйте складність.

Геймплейна роль: Wave generator створює macro-rhythm: pressure -> коротке полегшення -> сильніший pressure. Діапазони випадковості повинні працювати всередині цього ритму, а не руйнувати його.

10. Weapon Polish

- Додайте Muzzle Flash.
- Додайте Recoil.
- Додайте Shooting Sound.

Геймплейна роль: Weapon feedback формує perceived power. Recoil і muzzle flash повинні відповідати силі weapon: shotgun має відчуватися важчим за basic gun не лише числами.

11. Combat Polish

- Додайте Hit Effect.
- Додайте Enemy Death Effect.
- Додайте Camera Shake або інший feedback.

Геймплейна роль: Combat polish допомагає швидко читати outcome пострілу і робить repeated shooting satisfying. Camera shake варто дозувати: він має підсилювати сильні події, а не заважати aim.

12. Object Pool

- Реалізуйте Object Pool для Projectiles або Enemies.

Геймплейна роль: У wave shooter багато однотипних об'єктів регулярно створюються і зникають. Pooling робить систему стабільнішою технічно і готує її до більшої щільності бою.

Фінальний чекліст

- ☐ Пункти 1-6 працюють як замкнений gameplay loop.
- ☐ Виконана одна призначена гілка: А або В.
- ☐ Виконані пункти 10-12.
- ☐ Немає критичних помилок у Console під час звичайного gameplay.
- ☐ Ключові gameplay-параметри винесені в Inspector або data assets, де це доречно.
- ☐ Win/Game Over/Restart працюють без перезавантаження Editor.
- ☐ Студент може пояснити призначення кожного власного або AI-assisted компонента та його зв'язки з іншими компонентами.

Критерій готовності: Проєкт вважається працездатним не тоді, коли в ньому багато окремих mechanics, а коли всі реалізовані системи утворюють зрозумілий повторюваний loop із feedback, outcome та можливістю почати нову спробу.